

API 명세서

☼ 상태	준비
≡ 담당 부서	백엔드
≡ 문서 성격	
≡ 기술	
👤 생성자	 소연

[Farm-ily] API 및 프로토콜 상세 명세서

공통 사항

- **Base URL:** `https://api.family.site` (예시)
- **인증 방식:** Bearer Token (JWT) 헤더 사용
 - `Authorization: Bearer {ACCESS_TOKEN}`
- **날짜 포맷:** `YYYY-MM-DD HH:mm:ss` (KST 기준)
- **응답 공통 규격:**

```
{
  "code": 200,
  "message": "성공하였습니다.",
  "data": { ... }
}
```

1. HTTP REST API (Spring Boot Server)

A. 인증 (Auth)

Method	URI	설명	Request Body / Params	Response Data
POST	<code>/auth/email-req</code>	인증코드 발송 (앱)	<code>{ "email": "... " }</code>	<code>{ "expireTime": "180" }</code>

POST	/auth/email-verify	인증코드 검증 (앱)	{ "email": "...", "code": "... " }	{ "isVerified": true }
POST	/users/signup	회원가입 (앱)	{ "email": "...", "password": "... " }	{ "userId": 1 }
POST	/users/login	로그인 (앱)	{ "email": "...", "password": "... " }	{ "accessToken": "...", "refreshToken": "..." }
POST	/users/refresh	토큰 재발급 (앱)	(Header: Refresh Token)	{ "accessToken": "..." }
PATCH	/users/password	비밀번호 재설정 (앱)	{ "email": "...", "newPassword": "..." }	200 OK
DELETE	/users	회원 탈퇴 (앱)	(Header: Access Token)	200 OK

B. 식물 및 기기 (Plant & Device)

Method	URI	설명	Request Body / Params	Response Data
POST	/plants	식물 등록 (앱)	{ "species": "MONSTERA", "nickname": "두리", "uuid": "robot-001" }	{ "plantId": 1 }
GET	/home	메인 화면 정보 조회 (앱) (D+Day, 포인트, 등급)	(Header: Access Token)	{ "nickname": "두리", "dDay": 100, "points": 50.5, "grade": "LEVEL_3", "gradeName": "정 이 드는 사이" }
GET	/robots/config	부팅 시 상태 동기화 (Jetson)	?uuid=robot-001	{ "nickname": "두리", "optimalMoisture": 60, "status": "DEFAULT", **"gradeName": "정 이 드는 사이"** }
POST	/plants/pledge	치료 서약서 작성 (앱)	{ "plantId": 1, "diseaseCode": "LEAF_BLIGHT" }	{ "immuneUntil": "..." }

C. 센서 및 로그 (Sensor & Logs)

※ RPi 5 → Server (데이터 저장용)

Method	URI	설명	Request Body	Response
--------	-----	----	--------------	----------

POST	/sensors/logs	1시간 통계 저장 (RPi)	{ "uuid": "...", "temp": 24.5 ... }	200 OK
POST	/alerts/water	물 부족 알림 요청 (RPi)	{ "uuid": "...", "currentMoisture": 10 }	200 OK
POST	/alerts/return	복귀 알림 요청 (RPi)	{ "uuid": "...", "currentLight": 50 }	200 OK
POST	/interactions/talk	대화/칭찬 포인트 적립 (Jetson) (대화 종료 시 호출)	{ "uuid": "...", "type": "TALK" } 또는 { "type": "PRAISE" }	200 OK (+0.02 점)
POST	/robots/move	이동 명령 (앱) (앱 리모컨 기능)	{ "uuid": "...", "target": "STATION" }	200 OK (MQTT 발행 트리거)

D. AI 진단 및 앨범 (Diagnosis & Album)

Method	URI	설명	Request Body (Multipart)	Response
POST	/diagnosis	질병 진단 결과 (Jetson → Server)	file : (이미지파일) json : { "uuid": "...", "disease": "LEAF_SPOT", "confidence": 0.92 }	200 OK
POST	/photos/timelapse	정기 촬영 업로드 (Station → Server)	file : (이미지파일) json : { "uuid": "..." }	200 OK
POST	/diaries	육아 일기 작성 (앱 → Server)	file : (이미지파일) json : { "content": "오늘 두리가...", "weather": "SUNNY" }	200 OK (+0.08 점 지급)

2. 📡 MQTT Protocol (Real-time Communication)

로봇 내부(RPi ↔ Jetson) 및 로봇과 서버 간의 실시간 통신 규약입니다.

- **Broker:** AWS EC2 (Mosquitto)
- **QoS:** 1 (At least once)

A. 로봇 제어 및 설정 (Control & Config)

Topic	Pub	Sub	Payload (JSON)	설명
-------	-----	-----	----------------	----

robot/{uuid}/config	Server	Jetson	{ "nickname": "두리", "gradeName": "반려 사이", ... }	식물 등록/수정 또는 등급 변경 시 업데이트
robot/{uuid}/weather	Server	Jetson	{ "weather": "RAIN" }	IP 기반 날씨 정보 전송
robot/{uuid}/move	Server	Jetson	{ "target": "STATION" }	앱에서 이동 명령 시 서버가 발행
station/{uuid}/light	RPi	Station	{ "cmd": "ON" }	조명 제어

B. 센서 및 감정 표현 (Sensor & Emotion)

Topic	Pub	Sub	Payload (JSON)	설명
robot/{uuid}/sensor	RPi	App	{ "temp": 24, "soil": 60 ... }	1초 주기 실시간 데이터
robot/{uuid}/emotion	RPi/Svr	Jetson	{ "type": "TOUCH" } (쓰다듬기) { "type": "SICK" } (질병/물부족) { "type": "DEFAULT" }	표정 변화 트리거

3. 🧩 데이터 구조체 상세 (Data Structure)

개발 시 헷갈리지 않도록 주요 데이터 값의 범위를 정의합니다.

A. 질병 코드 (Disease Code)

코드명	의미	비고
HEALTHY	정상	알림 없음

B. 애착 등급 (Relationship Grade)

등급명 (Kor)	포인트 구간	DB 저장 값 (Enum)
어색한 사이	0 ~ 29.99	LEVEL_1
익숙해지는 사이	30 ~ 49.99	LEVEL_2
정이 드는 사이	50 ~ 69.99	LEVEL_3
마음을 여는 사이	70 ~ 89.99	LEVEL_4
반려 사이	90 ~ 100	LEVEL_5

4. ⚠ 예외 처리 및 엣지 케이스 (Edge Cases)

이 로직은 반드시 코드에 포함되어야 합니다.

1. 오프라인 큐 (Offline Queue) - RPi 5

- **상황:** Wi-Fi가 끊겼을 때 센서 데이터 유실 방지.
- **구현:** Python `deque` 사용. MQTT 전송 실패 시 데이터를 큐에 `append`. 재연결 (`on_connect`) 시 큐에 있는 데이터를 루프 돌려 전송.

2. 명령 잠금 (Command Lock) - Jetson

- **상황:** 12시 촬영 시간에 사용자가 로봇을 부를 때.
- **로직:** `11:59:00` ~ `12:01:00` 사이에는 이동 명령(Navigation) 수신 시 무시하고, TTS로 "사진 찍을 시간이에요" 출력.

3. 질병 면역 (Immunity) - Server

- **상황:** 잎마름병 진단 수신.
- **로직:** DB의 `Pledge` 테이블 조회.
 - `WHERE plant_id=? AND disease_code='LEAF_BLIGHT' AND created_at > NOW() - 3 DAYS`
 - 결과가 **있으면:** 알림 안 보냄 (Pass).
 - 결과가 **없으면:** 푸시 알림 발송 & MQTT `SICK` 발행.